COER UNIVERSITY

ROORKEE

Department of Computer Science & Engineering

LABORATORY FILE

Subject / Course	Full Stack Web Development
Experiment No.	06
Experiment Title	APIs — Node.js + Express REST + Clients
Student Name	Utsav Kumar Kushwaha
Roll No.	63
CU-ID.	CU240251668
Semester	V
Faculty Name	Ms. ANKITA
Submission Date	

Academic Year 2026–27

Experiment No. 06

Title: Full Backend Integration — Node.js + Express REST APIs (No Database)

Aim

To build a complete Node.js + Express REST API with in-memory CRUD, middleware (logger, API key auth), CORS, and error handling, then consume it from a plain JavaScript client and a React client with loading/error states and create/delete UI — covering the full backend-to-frontend pipeline without a database.

Theory

A REST API exposes resources over HTTP using standard methods (GET, POST, PUT, DELETE). Express provides routing and middleware. CORS allows browsers to call APIs from other origins. API keys on mutating routes add a simple security layer. On the client, fetch or axios retrieves data; React useEffect + useState implement the loading/error/data pattern. Environment variables keep secrets and base URLs out of source code.

Software Required

- Node.js, npm
- express, cors, dotenv, nodemon (server)
- Vite + React + axios (React client)
- Postman / Thunder Client / curl
- Web browser

Procedure

1. Create api-lab/server, npm init, install express cors dotenv; add .env with PORT and API_KEY.
2. Implement in-memory tasks array and five CRUD routes with 404 for missing ids.
3. Add global logger middleware and requireApiKey on POST/PUT/DELETE.
4. Enable CORS; test with and without it from a browser origin.
5. Add 404 handler and centralized 4-argument error middleware.
6. Build client/ with index.html + script.js (fetch, loading, error).
7. Build client-react/ Task Manager with useEffect, form POST, delete, VITE_API_URL.
8. Test all endpoints with Postman; verify 401 without key and graceful UI errors.

Source Code

Q1–Q2 — Server (server/index.js)

server/index.js

```
/**
 * API Lab – Lab Sheet 6
 * Express REST API: CRUD tasks, logger, auth, CORS, error handling
 */
require("dotenv").config();
const express = require("express");
const cors = require("cors");


const app = express();
const PORT = process.env.PORT || 5000;
const API_KEY = process.env.API_KEY || "lab-secret-key-123";


// ---------- In-memory tasks (at least 5) ----------
let tasks = [
  { id: 1, title: "Learn Express routing", completed: true },
  { id: 2, title: "Build REST CRUD API", completed: false },
  { id: 3, title: "Add CORS and middleware", completed: false },
  { id: 4, title: "Connect React client", completed: false },
  { id: 5, title: "Test with Postman", completed: true }
];
let nextId = 6;


// ---------- Middleware ----------
// CORS for development (Q2a)
app.use(cors());


// Body parsers
app.use(express.json());


// Custom logger – global (Q1c)
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});


// Auth check – only for mutating routes (Q1c)
function requireApiKey(req, res, next) {
  const key = req.headers["x-api-key"];
  if (!key || key !== API_KEY) {
    return res.status(401).json({ error: "Unauthorized – valid x-api-key header required" });
  }
  next();
}


// ---------- CRUD Routes (Q1b) ----------
// GET all
app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});


// GET one
app.get("/api/tasks/:id", (req, res) => {
  const task = tasks.find((t) => t.id === parseInt(req.params.id, 10));
  if (!task) {
    return res.status(404).json({ error: "Task not found" });
  }
  res.json(task);
});


// POST create (protected)
app.post("/api/tasks", requireApiKey, (req, res) => {
  const { title, completed } = req.body;
  if (!title || typeof title !== "string") {
    return res.status(400).json({ error: "title is required" });
  }
  const task = {
    id: nextId++,
    title: title.trim(),
    completed: Boolean(completed)
  };
  tasks.push(task);
  res.status(201).json(task);
});
```

```
// PUT update (protected)
app.put("/api/tasks/:id", requireApiKey, (req, res) => {
  const idx = tasks.findIndex((t) => t.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    return res.status(404).json({ error: "Task not found" });
  }
  const { title, completed } = req.body;
  if (title !== undefined) tasks[idx].title = String(title).trim();
  if (completed !== undefined) tasks[idx].completed = Boolean(completed);
  res.json(tasks[idx]);
});


// DELETE (protected)
app.delete("/api/tasks/:id", requireApiKey, (req, res) => {
  const idx = tasks.findIndex((t) => t.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    return res.status(404).json({ error: "Task not found" });
  }
  const removed = tasks.splice(idx, 1)[0];
  res.json({ message: "Task deleted", task: removed });
});


// ---------- 404 + centralized error handler (Q2c) ----------
app.use((req, res) => {
  res.status(404).json({ error: "Route not found" });
});


app.use((err, req, res, next) => {
  console.error("Error:", err.message);
  res.status(err.status || 500).json({
    error: err.message || "Internal Server Error"
  });
});


app.listen(PORT, () => {
  console.log(`API Lab server running on http://localhost:${PORT}`);
  console.log(`API key for POST/PUT/DELETE: ${API_KEY}`);
});
```

Server config

server/.env

```
PORT=5000
API_KEY=lab-secret-key-123
```

server/package.json

```
{
  "name": "api-lab-server",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

Q3 — Plain JavaScript Client

client/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>API Lab — Plain JS Client</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 560px; margin: 2rem auto; padding: 0 1rem; }
    h1 { color: #1a73e8; }
    #status { color: #5f6368; margin: 1rem 0; }
    #error { color: #d93025; font-weight: 600; }
    ul { padding-left: 1.2rem; }
    li { margin: 0.4rem 0; }
  </style>
</head>
<body>
  <h1>Task List (Plain JS)</h1>
  <div id="status">Loading...</div>
  <div id="error"></div>
  <ul id="list"></ul>
  <script src="script.js"></script>
</body>
</html>
```

client/script.js

```
// client/script.js — Plain JavaScript consumer (Q3)
const API_URL = "http://localhost:5000/api/tasks";


const statusEl = document.getElementById("status");
const errorEl = document.getElementById("error");
const listEl = document.getElementById("list");


async function loadTasks() {
  statusEl.textContent = "Loading...";
  errorEl.textContent = "";
  listEl.innerHTML = "";


  try {
    const res = await fetch(API_URL);
    if (!res.ok) {
      throw new Error(`Server error: ${res.status} ${res.statusText}`);
    }
    const tasks = await res.json();
    statusEl.textContent = `${tasks.length} task(s) loaded`;
    tasks.forEach((t) => {
      const li = document.createElement("li");
      li.textContent = `${t.title} ${t.completed ? "✓" : ""}`;
      listEl.appendChild(li);
    });
  } catch (err) {
    statusEl.textContent = "";
```

```
    // Clear message when server is stopped (Q3c)
    errorEl.textContent = "Failed to load tasks. Is the API server running? — " + err.message;
  }
}

loadTasks();
```

Q4–Q5 — React Client

client-react/src/App.jsx

```
/**
 * Task Manager Mini App — React client (Q4 + Q5)
 * Fetches, creates, deletes tasks against Express API
 */
import { useState, useEffect } from "react";
import axios from "axios";


const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:5000";
const API_KEY = "lab-secret-key-123"; // matches server .env for lab demo


function App() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [title, setTitle] = useState("");
  const [submitting, setSubmitting] = useState(false);


  // ----- Q4a: 3-state pattern with fetch -----
  async function loadTasks() {
    setLoading(true);
    setError(null);
    try {
      const res = await fetch(`${API_BASE}/api/tasks`);
      if (!res.ok) {
        const body = await res.json().catch(() => ({}));
        throw new Error(body.error || `HTTP ${res.status}`);
      }
      const data = await res.json();
      setTasks(data);
    } catch (err) {
      setError(err.message || "Failed to load tasks");
    } finally {
      setLoading(false);
    }
  }


  /*
   * Q4b — same call with axios (note differences):
   * - axios auto-parses JSON (no .json() call)
   * - errors throw on non-2xx; message often in err.response.data
   * - example:
   *   const { data } = await axios.get(`${API_BASE}/api/tasks`);
   *   setTasks(data);
   */


  useEffect(() => {
    loadTasks();
  }, []);


  // ----- Q4c: POST new task -----
  async function handleAdd(e) {
    e.preventDefault();
    if (!title.trim()) return;
    setSubmitting(true);
    setError(null);
    try {
      const res = await fetch(`${API_BASE}/api/tasks`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "x-api-key": API_KEY
        },
        body: JSON.stringify({ title: title.trim(), completed: false })
      });
      const body = await res.json().catch(() => ({}));
      if (!res.ok) {
        // Q5a: show 401/404 messages in UI
        throw new Error(body.error || `HTTP ${res.status}`);
      }
      setTasks((prev) => [...prev, body]);
      setTitle("");
    } catch (err) {
      setError(err.message);
    } finally {
      setSubmitting(false);
    }
  }


  // ----- Q4c: DELETE task -----
  async function handleDelete(id) {
```

```
    setError(null);
    try {
      const res = await fetch(`${API_BASE}/api/tasks/${id}`, {
        method: "DELETE",
        headers: { "x-api-key": API_KEY }
      });
      const body = await res.json().catch(() => ({}));
      if (!res.ok) {
        // e.g. fake id 9999 → 404 shown in UI (Q5a)
        throw new Error(body.error || `HTTP ${res.status}`);
      }
      setTasks((prev) => prev.filter((t) => t.id !== id));
    } catch (err) {
      setError(err.message);
    }
  }


  return (
    <div className="app">
      <h1>Task Manager Mini App</h1>
      <p className="sub">Express API + React (Lab Sheet 6)</p>


      {error && <div className="error-box">{error}</div>}


      <form className="add-form" onSubmit={handleAdd}>
        <input
          value={title}
          onChange={(e) => setTitle(e.target.value)}
          placeholder="New task title"
          disabled={submitting}
        />
        <button type="submit" disabled={submitting || !title.trim()}>
          {submitting ? "Adding..." : "Add Task"}
        </button>
      </form>


      {loading && <p className="status">Loading...</p>}


      {!loading && !error && tasks.length === 0 && (
        <p className="status">No tasks yet. Add one above.</p>
      )}


      <ul className="task-list">
        {tasks.map((t) => (
          <li key={t.id}>
            <span className={t.completed ? "done" : ""}>{t.title}</span>
            <button className="del" onClick={() => handleDelete(t.id)}>
              Delete
            </button>
          </li>
        ))}
      </ul>


      <button type="button" className="reload" onClick={loadTasks}>
        Reload
      </button>
    </div>
  );
}

export default App;
```

client-react/.env

```
VITE_API_URL=http://localhost:5000
```

Output (API & Client Behaviour)

Sample server responses (node index.js, then curl/Postman):

```
GET /api/tasks
[
  { "id": 1, "title": "Learn Express routing", "completed": true },
  { "id": 2, "title": "Build REST CRUD API", "completed": false },
  ...
]


GET /api/tasks/1
{ "id": 1, "title": "Learn Express routing", "completed": true }


GET /api/tasks/99
{ "error": "Task not found" }


POST /api/tasks  (without x-api-key)
{ "error": "Unauthorized — valid x-api-key header required" }


POST /api/tasks  (with x-api-key: lab-secret-key-123)
Body: { "title": "Write lab report", "completed": false }
Response 201: { "id": 6, "title": "Write lab report", "completed": false }


DELETE /api/tasks/6  (with key)
{ "message": "Task deleted", "task": { ... } }


GET /unknown
{ "error": "Route not found" }


CORS disabled (browser console):
Access to fetch at 'http://localhost:5000/api/tasks' from origin 'http://localhost:5173'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header...


After app.use(cors()) — requests succeed.
```

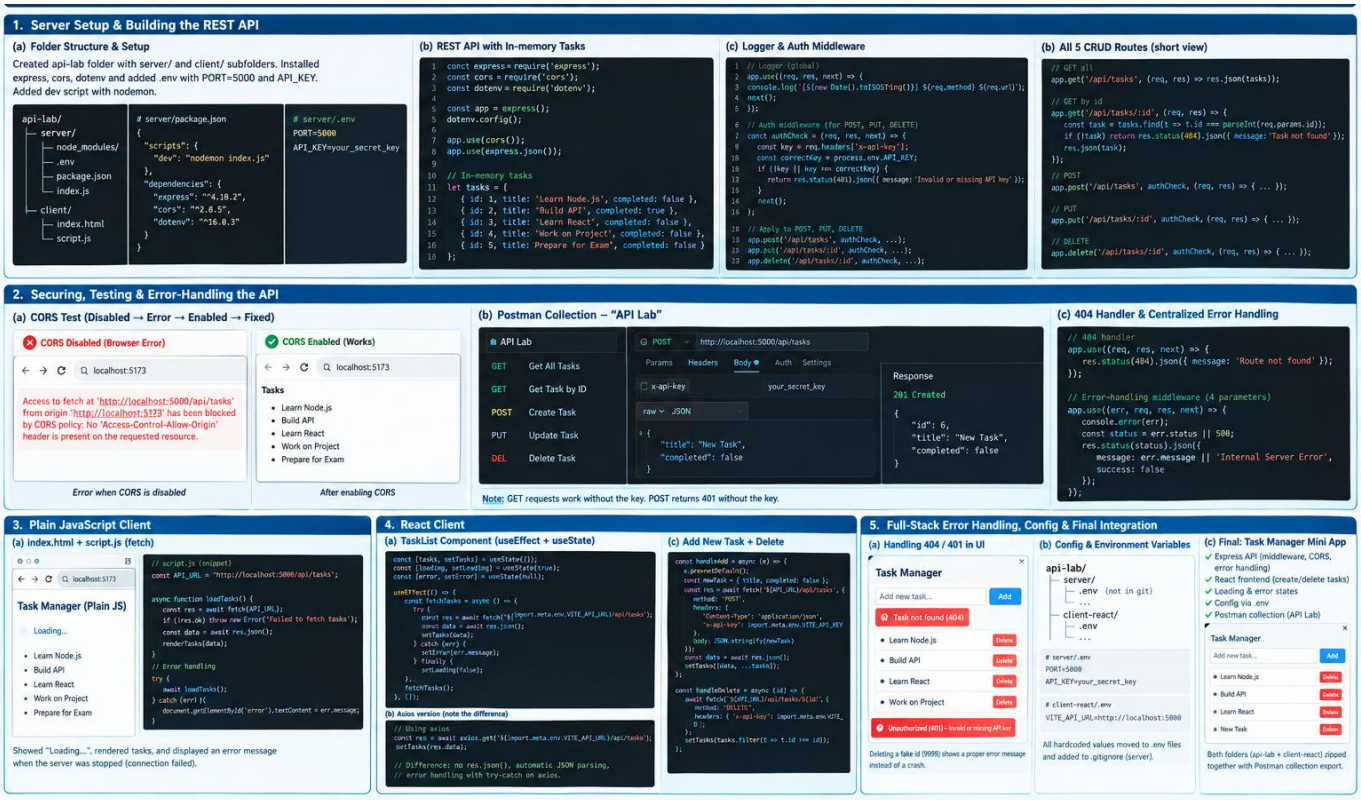
Plain JS client: shows “Loading...”, then task titles; if server is stopped, shows a clear error instead of crashing.

React client: loading / error / data states; Add Task (POST with x-api-key) updates list without reload; Delete removes item; fake id 9999 shows “Task not found” in the UI.

Result

A complete Task Manager stack was implemented: Express REST API with logger, API-key protection on writes, CORS, 404 and centralized errors; a plain JS fetch client with loading/error UI; and a React client using environment-based API URL, create/delete, and graceful handling of 401/404. All Lab Sheet 6 requirements (Q1–Q5) are covered.

OUTPUT :


1. Server Setup & Building the REST API
(a) Folder Structure & Setup
Created api-lab folder with server/ and client/ subfolders. Installed express, cors, dotenv and added .env with PORT=5000 and API_KEY. Added dev script with nodemon.
api-lab/
server/
node_modules/
.env
package.json
index.js
client/
index.html
script.js
server/package.json
{
"scripts": {
"dev": "nodemon index.js"
},
"dependencies": {
"express": "^4.18.2",
"cors": "^2.8.5",
"dotenv": "^16.0.3"
}
}
server/.env
PORT=5000
API_KEY=your_secret_key
(b) REST API with In-memory Tasks
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const app = express();
dotenv.config();
app.use(cors());
app.use(express.json());
// In-memory tasks
let tasks = [
{ id: 1, title: 'Learn Node.js', completed: false },
{ id: 2, title: 'Build API', completed: true },
{ id: 3, title: 'Learn React', completed: false },
{ id: 4, title: 'Work on Project', completed: false },
{ id: 5, title: 'Prepare for Exam', completed: false }
];
(c) Logger & Auth Middleware
// Logger (global)
app.use((req, res, next) => {
console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
next();
});
// Auth middleware (for POST, PUT, DELETE)
const authCheck = (req, res, next) => {
const key = req.headers['x-api-key'];
const correctKey = process.env.API_KEY;
if (!key || key !== correctKey) {
return res.status(401).json({ message: 'Invalid or missing API key' });
}
next();
};
// Apply to POST, PUT, DELETE
app.post('/api/tasks', authCheck, ...);
app.put('/api/tasks/:id', authCheck, ...);
app.delete('/api/tasks/:id', authCheck, ...);
(b) All 5 CRUD Routes (short view)
// GET all
app.get('/api/tasks', (req, res) => res.json(tasks));
// GET by id
app.get('/api/tasks/:id', (req, res) => {
const task = tasks.find(t => t.id === parseInt(req.params.id));
if (!task) return res.status(404).json({ message: 'Task not found' });
res.json(task);
});
// POST
app.post('/api/tasks', authCheck, (req, res) => { ... });
// PUT
app.put('/api/tasks/:id', authCheck, (req, res) => { ... });
// DELETE
app.delete('/api/tasks/:id', authCheck, (req, res) => { ... });
2. Securing, Testing & Error-Handling the API
(a) CORS Test (Disabled → Error → Enabled → Fixed)
CORS Disabled (Browser Error)
localhost:5173
Access to fetch at 'http://localhost:5000/api/tasks' from origin 'http://localhost:5173' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
Error when CORS is disabled
CORS Enabled (Works)
localhost:5173
Tasks
Learn Node.js
Build API
Learn React
Work on Project
Prepare for Exam
After enabling CORS
(b) Postman Collection – "API Lab"
API Lab
GET Get All Tasks
GET Get Task by ID
POST Create Task
PUT Update Task
DEL Delete Task
POST http://localhost:5000/api/tasks
Params Headers Body Auth Settings
x-api-key your_secret_key
raw JSON
{
"title": "New Task",
"completed": false
}
Response
201 Created
{
"id": 6,
"title": "New Task",
"completed": false
}
Note: GET requests work without the key. POST returns 401 without the key.
(c) 404 Handler & Centralized Error Handling
// 404 handler
app.use((req, res, next) => {
res.status(404).json({ message: 'Route not found' });
});
// Error-handling middleware (4 parameters)
app.use((err, req, res, next) => {
console.error(err);
const status = err.status || 500;
res.status(status).json({
message: err.message || 'Internal Server Error',
success: false
});
});
3. Plain JavaScript Client
(a) index.html + script.js (fetch)
localhost:5173
Task Manager (Plain JS)
Loading...
Learn Node.js
Build API
Learn React
Work on Project
Prepare for Exam
// script.js (snippet)
const API_URL = 'http://localhost:5000/api/tasks';
async function loadTasks() {
const res = await fetch(API_URL);
if (!res.ok) throw new Error('Failed to fetch tasks');
const data = await res.json();
renderTasks(data);
}
// Error handling
try {
await loadTasks();
} catch (err) {
document.getElementById('error').textContent = err.message;
}
Showed "Loading...", rendered tasks, and displayed an error message when the server was stopped (connection failed).
4. React Client
(a) TaskList Component (useEffect + useState)
const [tasks, setTasks] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
useEffect(() => {
const fetchTasks = async () => {
try {
const res = await fetch(`${import.meta.env.VITE_API_URL}/api/tasks`);
const data = await res.json();
setTasks(data);
} catch (err) {
setError(err.message);
} finally {
setLoading(false);
}
};
fetchTasks();
}, []);
(b) Axios version (note the difference)
// Using axios
const res = await axios.get(`${import.meta.env.VITE_API_URL}/api/tasks`);
setTasks(res.data);
// Difference: no res.json(), automatic JSON parsing,
// error handling with try-catch on axios.
(c) Add New Task + Delete
const handleAdd = async (e) => {
e.preventDefault();
const newTask = { title, completed: false };
const res = await fetch(`${API_URL}/api/tasks`, {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'x-api-key': import.meta.env.VITE_API_KEY
},
body: JSON.stringify(newTask)
});
const data = await res.json();
setTasks([data, ...tasks]);
};
const handleDelete = async (id) => {
await fetch(`${API_URL}/api/tasks/${id}`, {
method: 'DELETE',
headers: { 'x-api-key': import.meta.env.VITE_
});
setTasks(tasks.filter(t => t.id !== id));
};
5. Full-Stack Error Handling, Config & Final Integration
(a) Handling 404 / 401 in UI
Task Manager
Add new task...
Add
Task not found (404)
Learn Node.js
Build API
Learn React
Work on Project
Delete
Unauthorized (401) – Invalid or missing API key
Deleting a fake id (9999) shows a proper error message instead of a crash.
(b) Config & Environment Variables
api-lab/
server/
.env (not in git)
...
client-react/
.env
...
server/.env
PORT=5000
API_KEY=your_secret_key
client-react/.env
VITE_API_URL=http://localhost:5000
All hardcoded values moved to .env files and added to .gitignore (server).
(c) Final: Task Manager Mini App
Express API (middleware, CORS, error handling)
React frontend (create/delete tasks)
Loading & error states
Config via .env
Postman collection (API Lab)
Task Manager
Add new task...
Add
Learn Node.js
Build API
Learn React
New Task
Delete
Both folders (api-lab + client-react) zipped together with Postman collection export.